

Structural topic modeling (the 'stm' R package in (R))

INTRODUCTION TO NATURAL LANGUAGE PROCESSING

CSCI E-89B

Intro to Structural Topic Modeling in R

My favorite definition for STM is:

“The Structural Topic Model (STM) is a form of topic modelling specifically designed with social science research in mind. STM allow us to incorporate metadata into our model and uncover how different documents might talk about the same underlying topic using different word choices.”

Reference: <https://towardsdatascience.com/introduction-to-the-structural-topic-model-stm-34ec4bd5383>

STM Package

An R Package for the Structural Topic Model

Explanation: “The Structural Topic Model (STM) allows researchers to estimate topic models with document-level covariates. The package also includes tools for model selection, visualization, and estimation of topic-covariate regressions.”

R Documentation: <https://www.rdocumentation.org/packages/stm/versions/1.3.7>

Functions in STM & others

Functions in stm (1.3.7)

Search all functions

calcfrex
Calculate FREX (FREquency and EXclusivity) Words

calclift
Calculate Lift Words

convertCorpus
Convert stm formatted documents to another format

estimateEffect
Estimates regressions using an STM object

alignCorpus
Align the vocabulary of a new corpus to an old corpus

asSTMCorpus
STM Corpus Coercion

make.heldout
Heldout Likelihood by Document Completion

findTopic
Find topics that contain user specified words.

gadarian
Gadarian and Albertson data

js.estimate
A James-Stein Estimator Shrinking to a Uniform Distribution

plot.MultimodDiagnostic
Plotting Method for Multimodality Diagnostic Objects

exclusivity
Exclusivity

makeDesignMatrix
Make a Design Matrix

findThoughts
Find Thoughts

fitNewDocuments
Fit New Documents

plot.STM
Functions for plotting STM objects

calcscore
Calculate Score Words

plotRemoved
Plot documents, words and tokens removed at various word thresholds

STM Package Functionality

The package currently includes functionality to:

- ingest and manipulate text data
- estimate Structural Topic Models
- calculate covariate effects on latent topics with uncertainty
- estimate a graph of topic correlations
- compute model diagnostics and summary measures
- create all the plots used in our various techniques/examples

How to use your STM Package

Make sure to install the package:

- `install.packages("stm")`

Add your library as well to use this package:

- `library(stm)`

STM Package Detailed Information

Link to the package information: <https://www.structuraltopicmodel.com/>

A Great Example

This example walks you step-by-step through how to use the STM package:

- <https://github.com/dondealban/learning-stm>

LDA Versus STM

LDA versus STM example: <https://stackoverflow.com/questions/52948857/how-to-plot-a-document-topic-distribution-in-structural-topic-modeling-r-package>

Basic Demo using STM with a book!

Full example found here:

- <https://juliasilge.com/blog/sherlock-holmes-stm/>

Deeper level example with more words, training & evaluation:

- <https://juliasilge.com/blog/evaluating-stm/>

Note: We will walk through this next!

Let's walk through an Example here!

We will take the first link from the slide before: <https://juliasilge.com/blog/sherlock-holmes-stm/>

Download your text

```
library(tidyverse)
library(gutenbergr)

sherlock_raw <- gutenberg_download(1661)

sherlock <- sherlock_raw %>%
  mutate(story = ifelse(str_detect(text, "ADVENTURE"),
                        text,
                        NA)) %>%
  fill(story) %>%
  filter(story != "THE ADVENTURES OF SHERLOCK HOLMES") %>%
  mutate(story = factor(story, levels = unique(story)))

sherlock
```

Note: In R to assign a variable you can either use “<-” or “=”. I suggest you stick with one or the other.

Transform into TidyTex

```
library(tidytext)

tidy_sherlock <- sherlock %>%
  mutate(line = row_number()) %>%
  unnest_tokens(word, text) %>%
  anti_join(stop_words) %>%
  filter(word != "holmes")

tidy_sherlock %>%
  count(word, sort = TRUE)
```

Learning words that are important

```
library(drlib)

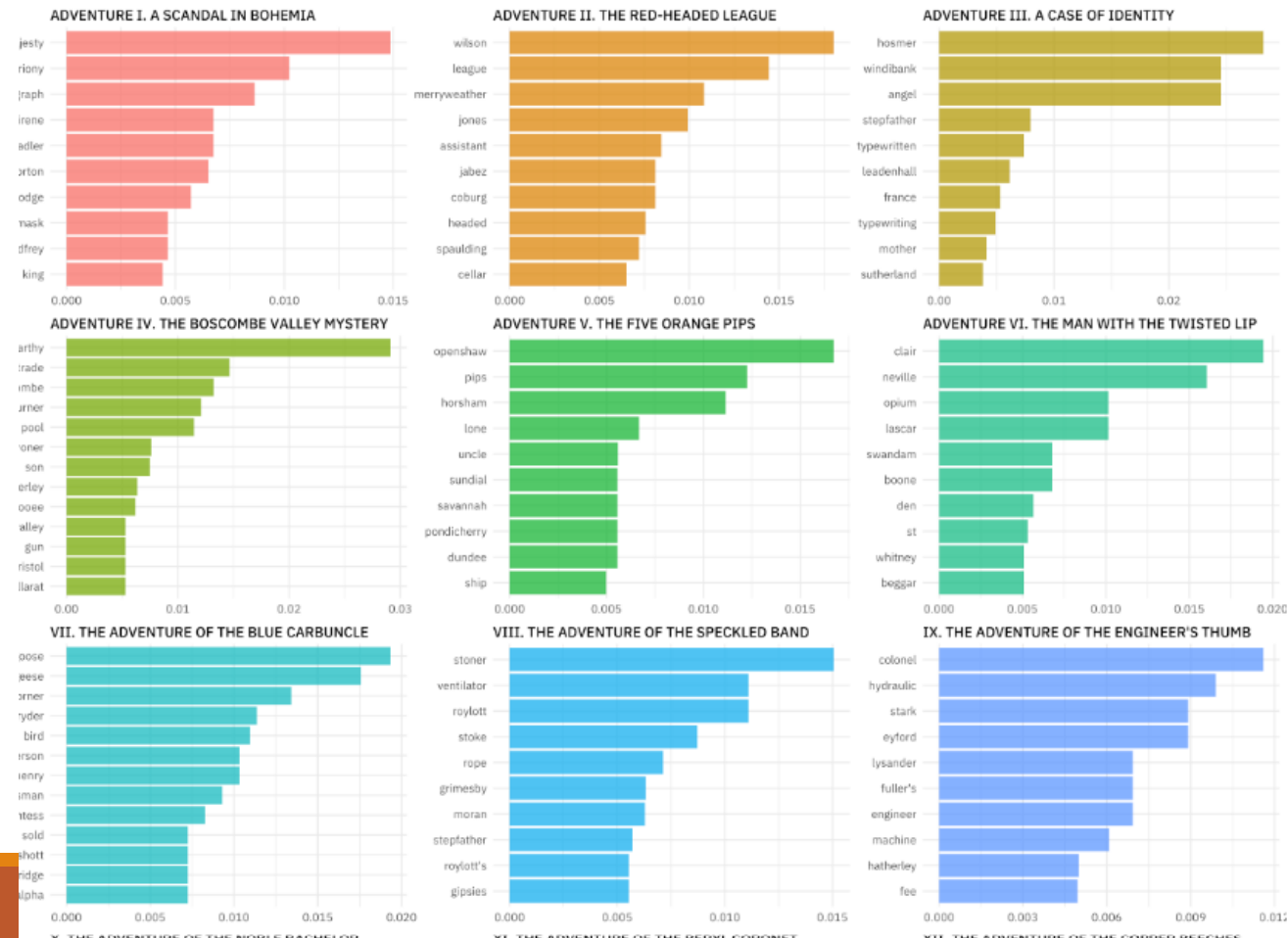
sherlock_tf_idf <- tidy_sherlock %>%
  count(word, story, sort = TRUE) %>%
  bind_tf_idf(word, story, n) %>%
  arrange(-tf_idf) %>%
  group_by(story) %>%
  top_n(10) %>%
  ungroup

sherlock_tf_idf %>%
  mutate(word = reorder_within(word, tf_idf, story)) %>%
  ggplot(aes(word, tf_idf, fill = story)) +
  geom_col(alpha = 0.8, show.legend = FALSE) +
  facet_wrap(~ story, scales = "free", ncol = 3) +
  scale_x_reordered() +
  coord_flip() +
  theme(strip.text=element_text(size=11)) +
  labs(x = NULL, y = "tf-idf",
       title = "Highest tf-idf words in Sherlock Holmes short stories",
       subtitle = "Individual stories focus on different characters and narr
```

Learning words that are important - Graph

Highest tf-idf words in Sherlock Holmes short stories

Individual stories focus on different characters and narrative elements



Topic Model

```
library(quanteda)
library(stm)

sherlock_dfm <- tidy_sherlock %>%
  count(story, word, sort = TRUE) %>%
  cast_dfm(story, word, n)

sherlock_sparse <- tidy_sherlock %>%
  count(story, word, sort = TRUE) %>%
  cast_sparse(story, word, n)
```


STM Function

```
topic_model <- stm(sherlock_dfm, K = 6,  
                  verbose = FALSE, init.type = "Spectral")
```

Plotting

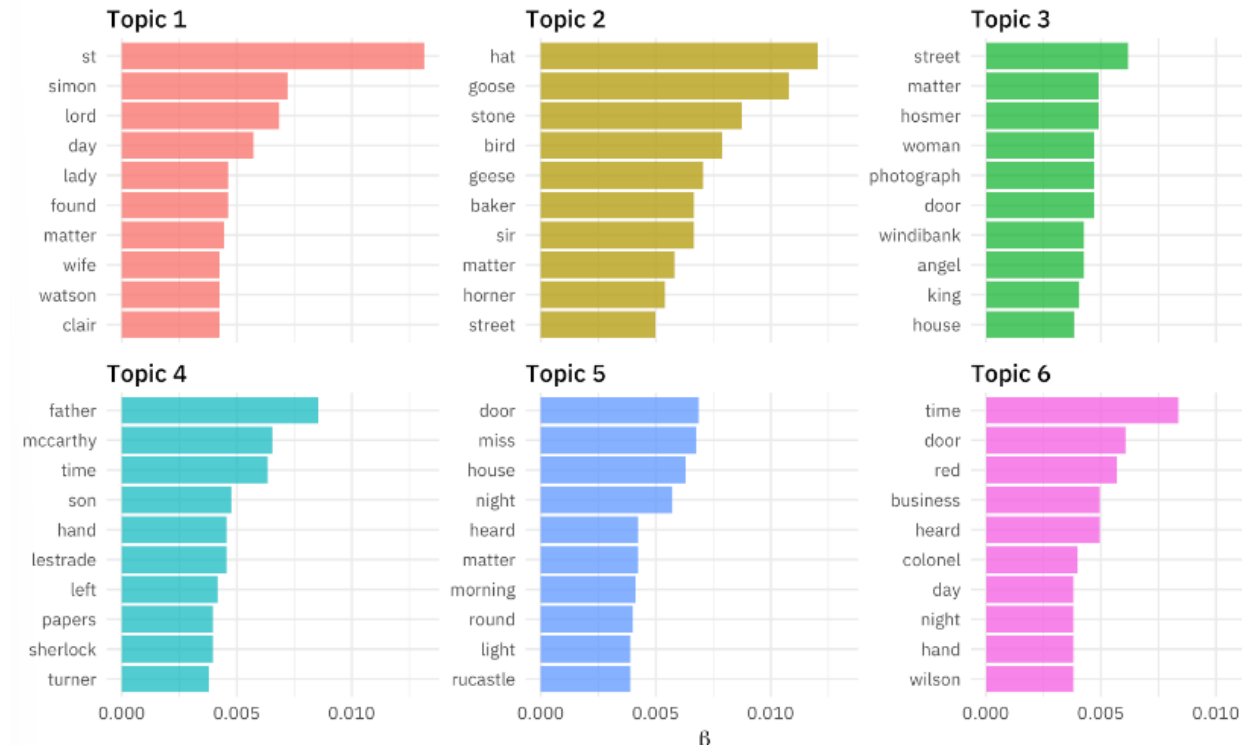
```
td_beta <- tidy(topic_model)

td_beta %>%
  group_by(topic) %>%
  top_n(10, beta) %>%
  ungroup() %>%
  mutate(topic = paste0("Topic ", topic),
         term = reorder_within(term, beta, topic)) %>%
  ggplot(aes(term, beta, fill = as.factor(topic))) +
  geom_col(alpha = 0.8, show.legend = FALSE) +
  facet_wrap(~ topic, scales = "free_y") +
  coord_flip() +
  scale_x_reordered() +
  labs(x = NULL, y = expression(beta),
       title = "Highest word probabilities for each topic",
       subtitle = "Different words are associated with different topics")
```

Highest Word Probabilities by Topic

Highest word probabilities for each topic

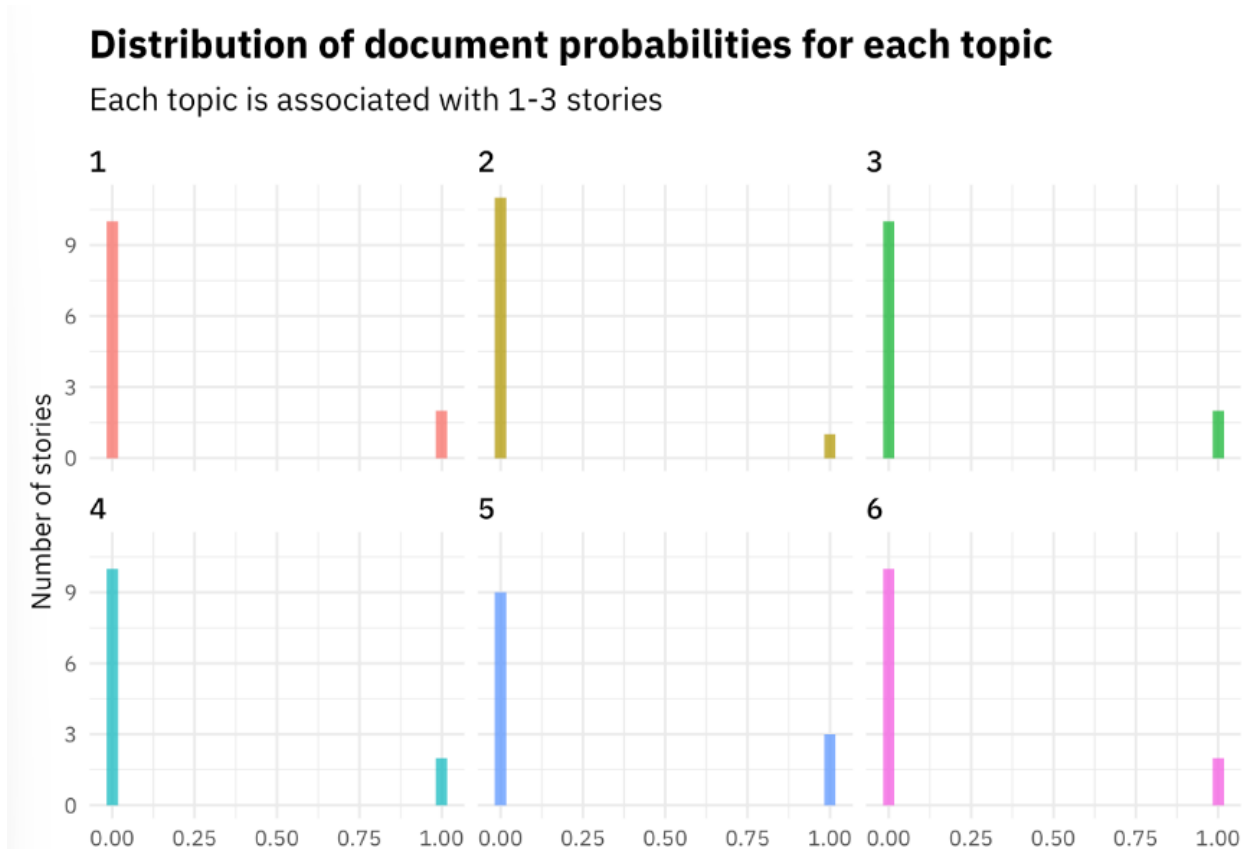
Different words are associated with different topics



Another Probability

```
td_gamma <- tidy(topic_model, matrix = "gamma",  
                 document_names = rownames(sherlock_dfm))  
  
ggplot(td_gamma, aes(gamma, fill = as.factor(topic))) +  
  geom_histogram(alpha = 0.8, show.legend = FALSE) +  
  facet_wrap(~ topic, ncol = 3) +  
  labs(title = "Distribution of document probabilities for each topic",  
        subtitle = "Each topic is associated with 1-3 stories",  
        y = "Number of stories", x = expression(gamma))
```

Graph of Document Probabilities



Exploration from the modeler's findings

- She built a small app to look at her models: <https://juliasilge.shinyapps.io/sherlock-holmes/>
- Like any model there are shifts that seem apparent!
- We have learned a lot and that using tidy data principles will help us

Basic Demo using STM with a book!

Demo with steps as seen here: <https://rpubs.com/hsuemrosen/basic-stm-demo>

Let's open this site and run through this page!

This is a pretty neat and well documented example!